The Array Geometry Handbook

How the Pinhole Array Design Code Works, and Why

IYPT Pinhole Sunglasses Project — A Student's Guide to the Physics and the Pipeline

August 16, 2026

Contents

1 Why This Folder Exists **2**
- 1.1 The Two Physical Effects That Matter . . . 2
- 1.2 What the Code Does, in One Sentence . . . 2

2 The Ghost-Separation Formula **3**
- 2.1 Building the Intuition First . . . 3
- 2.2 The Formula . . . 3
- 2.3 How the Correct Formula Was Actually Confirmed . . . 3

3 The Vignetting Model **4**
- 3.1 Why Ghosting Alone Is Not Enough . . . 4
- 3.2 The Geometry . . . 4
- 3.3 How the Implementation Was Checked . . . 5

4 The Nine Geometry Families **6**
- 4.1 Deterministic Families . . . 6
- 4.2 Stochastic (Randomized) Families . . . 6

5 The Central Finding: Ghost-Safe Does Not Mean Vignetting-Safe **7**

6 A Real Bug Worth Understanding: When "Density" Stops Making Sense **8**
- 6.1 The Problem . . . 8
- 6.2 The Fix, and the Lesson . . . 9

7 Choosing a Winner: The Two Comparison Metrics **11**
- 7.1 Metric 1: Maximum Packing Density . . . 11
- 7.2 Metric 2: The Pareto Front . . . 11

8 Closing the Loop: Confirming the Theory Against Real Physics **11**
- 8.1 The Confirmation Step . . . 11

9 Summary: What a Design Needs to Pass **12**

1 Why This Folder Exists

Imagine you have already solved the "easy" version of this problem: for a single pinhole, what diameter gives the sharpest possible image for someone with a given amount of short-sightedness? That question was answered earlier in this project, and the answer changes with myopia power M (in diopters) — a more short-sighted eye needs a smaller hole. That single-hole answer is stored in `new_sweeps/per_M_optimum_map.csv` and is treated as a *fixed input* to everything in this handbook: we are not re-deriving hole size here.

The harder question is: sunglasses need to block most of the light, and a single tiny hole lets almost none through. The natural fix is to use *many* small holes instead of one. But the moment you have more than one hole, a brand-new problem appears that a single hole never has: the eye can form more than one image of the same scene at once, one through each hole. If the eye is not perfectly focused (which, for someone wearing tinted glasses rather than corrective lenses, is basically always true to some degree), those images land in *slightly different places* on the retina. A small offset just looks like extra blur. A large offset looks like literal double vision — a real, disqualifying defect.

This handbook is about the code that answers one specific question: **given a fixed hole size (already chosen to be optimal), how should those many holes be arranged so that the double-vision problem never happens, for every level of myopia and every direction the eye might be looking?**

1.1 The Two Physical Effects That Matter

Physical Intuition

Two physically independent things can go wrong with a multi-hole array, and the code treats them as two separate checks, not one:

1. **Ghosting.** If two holes are far enough apart, and the eye is out of focus, the two copies of the image they produce land in different places on the retina — literally two overlapping pictures.

2. **Vignetting.** As the eye rotates to look around a scene, the pupil — a fixed-size circular opening — physically slides across the pattern of holes. Holes that were far from the pupil edge at straight-ahead gaze can suddenly come into play once the eye turns, changing *which* holes are simultaneously active and therefore changing the ghosting picture entirely.

A pattern that is perfectly safe when the eye looks straight ahead can become unsafe the moment the eye looks 15 degrees to one side. This is the single most important finding in the whole study, and it is why the code always checks both effects separately rather than assuming one implies the other.

1.2 What the Code Does, in One Sentence

For a chosen arrangement of holes (a *geometry*), at a chosen hole count and spacing, the code computes: (a) will any two holes ever produce a visible double image, at any myopia level, and (b) will that stay true as the eye looks around — and if both answers are "no", the design is *structurally safe*. It then compares many such safe designs against each other to find the one that packs the most holes (and therefore lets the most light through) without crossing either safety line.

2 The Ghost-Separation Formula

2.1 Building the Intuition First

Picture two pinholes, side by side, separated by a distance s (their center-to-center pitch). Light from a single point far away enters both holes. If the eye's lens is perfectly focused for that light, both holes' worth of light converges to *exactly the same spot* on the retina — this is just what "in focus" means: parallel rays entering a lens at different positions still meet at one point.

Now suppose the eye is short-sighted by M diopters — the lens converges light slightly too much (or too little) for that point to land exactly on the retina. The image plane is now displaced from the retina by a small amount related to M. Each hole, because it sits at a different lateral position, now projects its slightly-out-of-focus point to a *slightly different* spot on the retina. The two images are no longer perfectly overlapping.

Physical Intuition

The key physical fact that makes this predictable: the size of that retinal offset grows in direct proportion to *both* the hole separation s *and* how far out of focus the eye is (M). Zero separation, or zero defocus, means zero offset — there is nothing subtle here, it is basic similar-triangles geometry once you draw the ray paths through the eye's lens.

2.2 The Formula

The validated closed-form result used throughout this project is:

$$\Delta r = \frac{s \cdot M}{P_0 - M} \tag{1}$$

where s is the hole-to-hole separation, M is myopia in diopters, and P_0 is the eye's total optical power (a fixed constant of the eye model, ≈ 60D). Notice the formula automatically gives $\Delta r = 0$ when $M = 0$ — exactly the "in focus, both holes agree" case described above. This is not a coincidence; it is the formula correctly reproducing a case we already know the answer to by pure geometry, which is one of the strongest kinds of sanity check available in physics.

An earlier, uncorrected version of this formula was used in this project before it was properly checked. It did *not* go to zero at $M = 0$, which on its own was enough to know it was wrong — a formula that disagrees with a case you can reason out by hand needs no further testing to be rejected.

2.3 How the Correct Formula Was Actually Confirmed

Rather than trust either candidate formula on paper, this project built an independent test: place exactly two holes in a real wave-optics simulation (not a formula — an actual propagation of light through the eye model), measure where the two resulting bright spots land on the simulated retina, and see which formula's prediction matches. Figure 1 shows the result: the corrected formula fits the directly-measured data with $R^2 = 0.995$ (essentially perfect agreement), while the old formula scores $R^2 = -1701.7$ — a negative R^2 means the old formula was *worse than simply guessing the average value every time*, which is about as decisively wrong as a formula can be shown to be.

Where This Lives in the Code

The formula itself lives in `array_geometry_lab/degeneracy_filter.py` and `array_optimization/stage0_ghost_validation/` (the original validation, copied for speed into the geometry-comparison pipeline so it can be evaluated thousands of times without re-running full wave optics each time). The safety rule built on top of it is simple: for every pair of holes that could be simultaneously lit by the pupil, compute Δr; if the eye's own ability to resolve two separate points is smaller than every Δr in the pattern, the design is ghost-safe. This is `gate_ghost_safety()` in `comparison_metric/gates.py`.

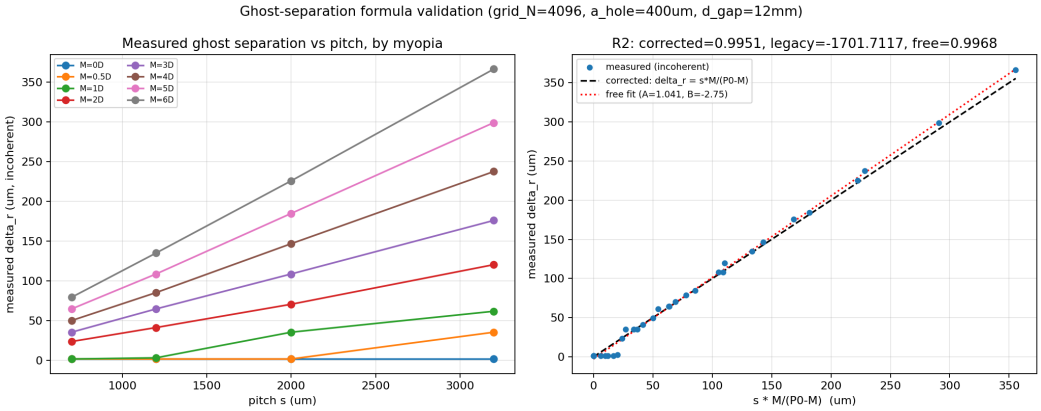


Figure 1: Directly-measured ghost separation (from real two-hole wave-optics propagation) versus both candidate formulas. The corrected formula (R^2=0.995) is the one used throughout this project's array code; the legacy formula ($R^2 = -1701.7$) is not.

3 The Vignetting Model

3.1 Why Ghosting Alone Is Not Enough

The ghost formula above assumes we already know *which pairs of holes* are simultaneously letting light through to the eye. When the eye looks straight ahead, the pupil sits centered behind the array, and it is easy to list which holes fall under it. But the pupil is a physical opening of fixed size, and the eye rotates constantly during normal viewing. As it rotates, the pupil's position relative to the fixed hole pattern shifts — some holes that were fully under the pupil become partially or fully blocked, and others that were outside the pupil's reach can come into play.

Physical Intuition

This means a design that looks perfectly safe — even *trivially* safe, with no two holes ever simultaneously active — when evaluated only at straight-ahead gaze can become unsafe the instant the eye looks even a little to one side. You cannot conclude a design is safe from the straight-ahead case alone; you have to check across the whole range of gaze angles a person actually uses.

3.2 The Geometry

The pupil and a given hole are both circles. As the eye rotates by an angle φ, the pupil's center shifts laterally relative to the hole pattern by approximately

$$\Delta(\varphi) \approx R_{\text{rot}} \sin\varphi \tag{2}$$

where R_{rot} is the eye's physiological rotation radius (≈13–14mm). The overlapping area between the pupil (radius R_p) and a hole (radius R_h), once their centers are separated by Δ, is the standard geometric formula for the overlap of two circles:

$$A_{\text{overlap}}(\Delta) = R_p^2 \cos^{-1}\left(\frac{\Delta^2 + R_p^2 - R_h^2}{2\Delta R_p}\right) + R_h^2 \cos^{-1}\left(\frac{\Delta^2 + R_h^2 - R_p^2}{2\Delta R_h}\right) - \frac{1}{2}\sqrt{(-\Delta+R_p+R_h)(\Delta+R_p-R_h)(\Delta+R_h- \tag{3}$$

with two edge cases handled separately: if the circles no longer touch at all ($\Delta \geq R_p + R_h$), the overlap is exactly zero; if one circle sits entirely inside the other ($\Delta \leq |R_p - R_h|$), the overlap is simply the area of the smaller circle. A naïve implementation of the general formula can return a tiny negative number right at this second boundary, purely from floating-point rounding — a real bug this project found and fixed by explicitly clamping the result to zero at the boundary.

3.3 How the Implementation Was Checked

The closed-form formula above was tested against a second, completely independent method: randomly scatter thousands of points inside a hole's area and simply count what fraction land inside the shifted pupil circle (a Monte Carlo estimate). If the two methods — one algebraic, one brute-force random sampling — agree, that is strong evidence the formula was implemented correctly, since the two approaches share no code and could not fail in the same way by coincidence. Figure 2 shows this comparison, converging to under 0.2 percentage points of relative error.

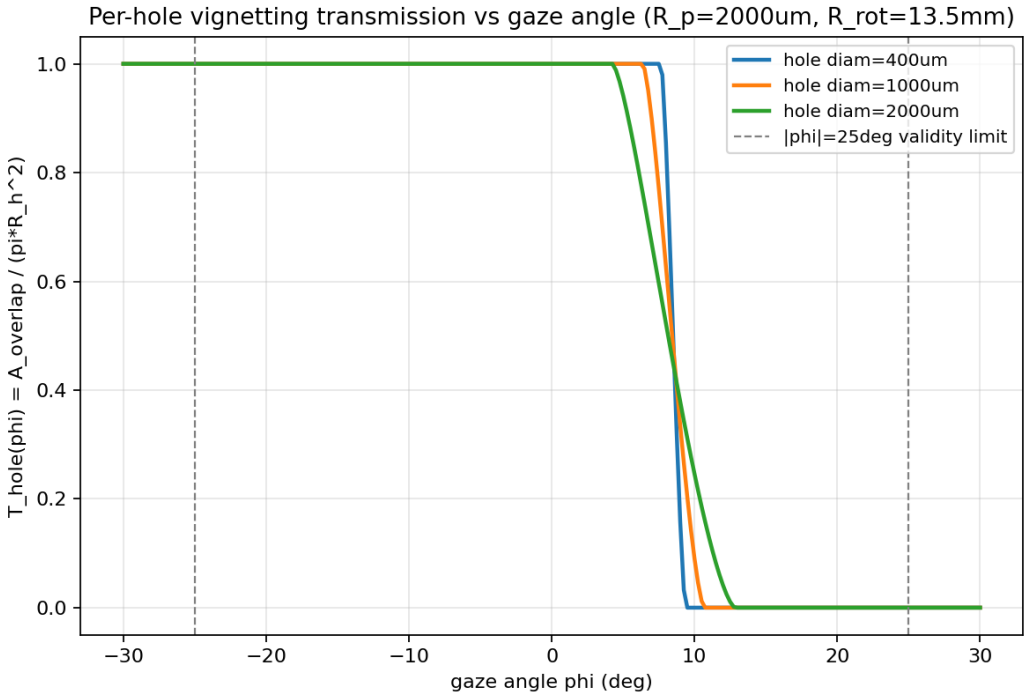


Figure 2: Closed-form overlap-area formula versus an independent Monte Carlo estimate, across gaze angle φ. Agreement here is what justifies trusting the fast closed-form version for large sweeps instead of running the (much slower) random-sampling method every time.

Where This Lives in the Code

This geometry lives in `array_optimization/vignetting.py` (the original, validated implementation) and is copied into `array_geometry_lab/degeneracy_filter.py`[a] for fast repeated use across the geometry-comparison sweep. The safety rule built on it, `gate_vignetting_safety()` in `comparison_metric/gates.py`, re-runs the "which holes are active" calculation at many gaze angles (0° to ±25°) and requires the design to stay ghost-safe at *every* one of them, not just at $\varphi = 0$.

[a] Copied rather than imported for the large sweep pipeline, for speed — but re-checked byte-for-byte against the original's own validation data before use, so the copy could not silently drift from the original.

4 The Nine Geometry Families

With the two physical checks defined, the natural next question is: does the *shape* of the hole pattern matter? A square grid, a honeycomb, rings, a spiral, or even a scattered random pattern all place N holes over the same area — but they place them in very different relative positions, which changes both the density you can achieve and the worst-case hole-to-hole distances that feed the ghost formula.

4.1 Deterministic Families

Five families produce the *same* layout every time, given the same inputs (hole count, spacing) — there is no randomness involved:

- **Square** and **Hexagonal** lattices — the simplest, most conventional packings. Hex packs more circles into a given area than square (about 90.7% of the area versus 78.5% at maximum density), which is why it tends to win on packing density when it survives the safety gates.
- **Rectangular** — a square lattice allowed to have different spacing in the x and y directions, motivated by the fact that natural eye movement is not symmetric (people saccade horizontally more than vertically).
- **Concentric rings** — holes arranged in circles around the center, a natural match to the pupil's own circular shape.
- **Fibonacci spiral / golden-angle phyllotaxis** — the pattern seen in sunflower seed heads, where each new point is placed at a fixed angular step (the golden angle, ≈137.5°) from the last. These two names describe the mathematically identical pattern; the code confirms this explicitly rather than accidentally shipping the same generator twice under different names.

4.2 Stochastic (Randomized) Families

Two families place holes at *different* positions every time they are generated, even with identical inputs, because they use randomness:

- **Poisson-disk ("blue-noise")** — random placement with a hard minimum-distance rule enforced between every pair of holes.
- **Jittered lattice** — a hexagonal lattice with each hole's position nudged by a small random offset.

The reason to test these at all: every deterministic family above is periodic (the pattern repeats at a fixed spacing), and periodic apertures are known in optics to produce secondary bright spots at predictable angles (diffraction grating orders) on top of whatever the ghost formula already predicts. Breaking the periodicity is one way to avoid that side effect entirely — but a random pattern's safety now has to be checked *across many random realizations (seeds)*, not just once, since the same requested parameters can produce a lucky safe layout on one seed and an unsafe one on the next.

Where This Lives in the Code

Every geometry family is a small, self-contained function in `array_geometry_lab/generators/`, one file per family, all sharing one interface: given a requested hole count and spacing, return a list of hole-center coordinates. Every generator also rejects (or removes) any hole placement that would physically overlap its neighbors — a fabrication constraint separate from and prior to the optical safety checks. This overlap-rejection can be aggressive: one documented test case showed a requested 50 holes collapsing to only 9 actual holes once overlap was enforced, at a spacing too tight for that hole size.

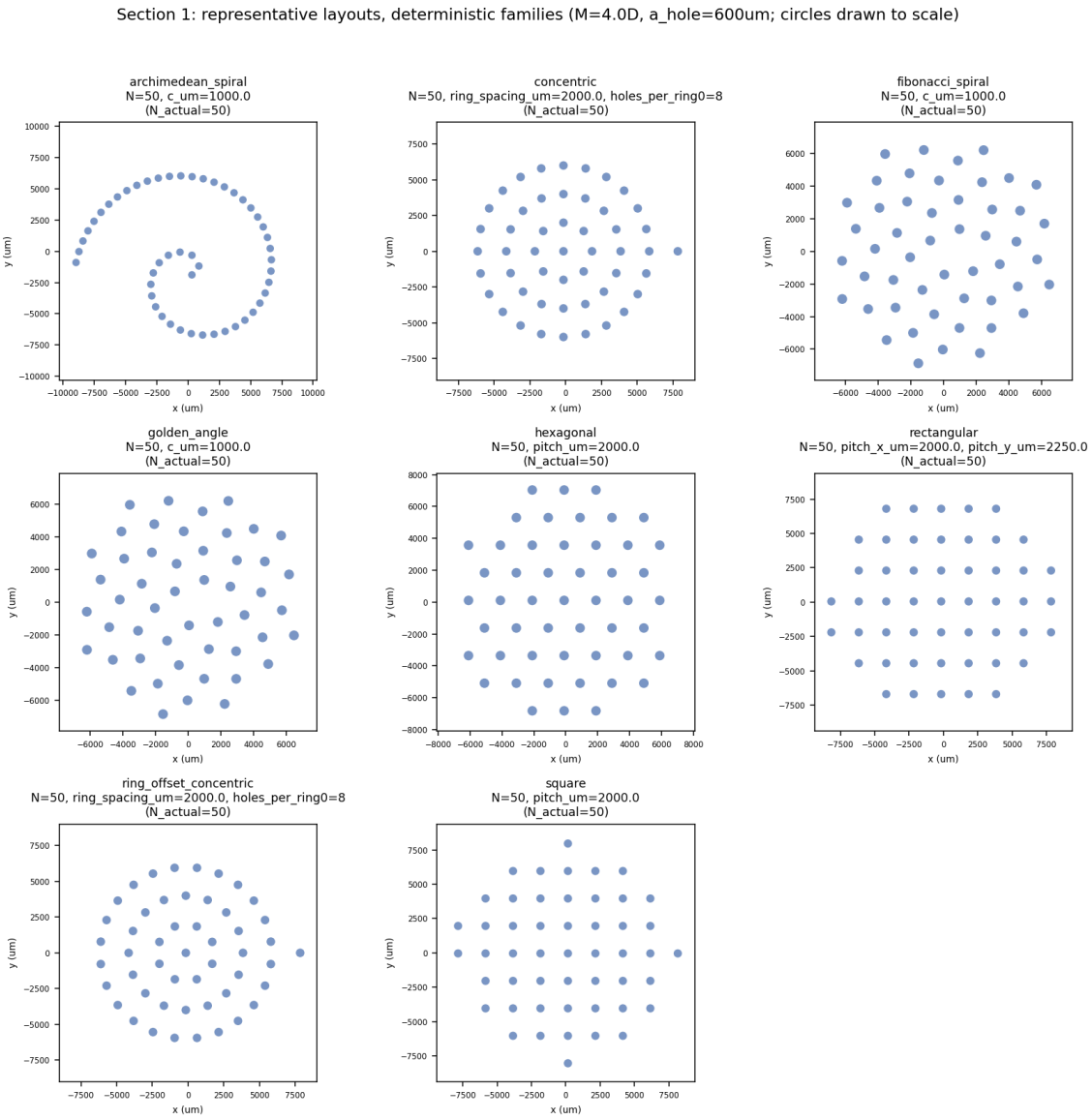


Figure 3: Representative generated layouts for the five deterministic families, drawn to scale (hole radius shown as filled circles, not just points) so packing density is visually honest.

5 The Central Finding: Ghost-Safe Does Not Mean Vignetting-Safe

This is the single most important experimental result to come out of the whole geometry comparison, and it is worth stating plainly before looking at any numbers: **a design can have an infinite ghost-safety margin at straight-ahead gaze (meaning literally no two holes are ever simultaneously active under the pupil) and still fail the vignetting check once the eye looks around.**

Physical Intuition

Why this makes physical sense: "no two holes active at once" at $\varphi = 0$ only describes one single eye position. As the pupil shifts during rotation, holes that were previously outside its reach can come into play together, creating an active pair that did not exist before. Zero risk at one gaze angle says nothing about zero risk at every other gaze angle — these genuinely are two separate conditions to check.

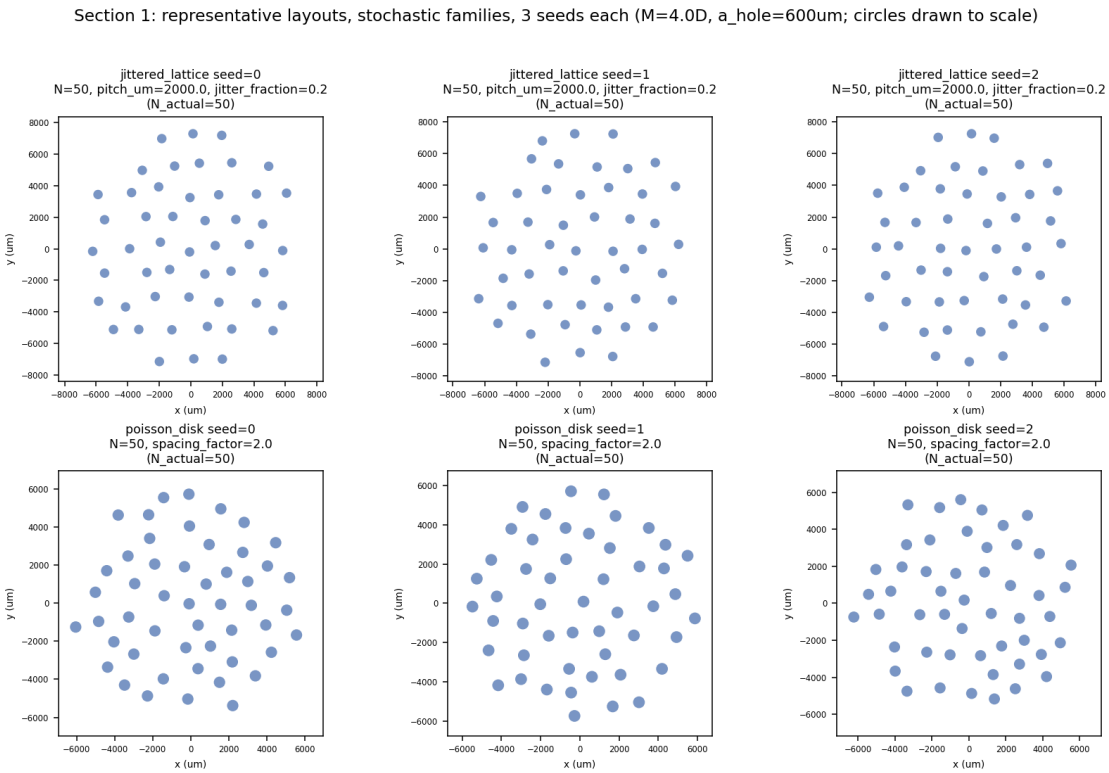


Figure 4: Three different random seeds for the two stochastic families, shown side by side. The visible differences between seeds are exactly why these families need seed-averaged safety checking rather than a single evaluation.

Figure 5 shows how often this actually happens, measured directly from the data (not assumed): for every family, the fraction of straight-ahead-safe designs that turn out to *fail* the vignetting check ranges from 37.8% (concentric rings, the most robust family) to 88.9% (Fibonacci spiral / golden angle, the least robust). This is not a small-print caveat — for the spiral families, roughly nine out of ten designs that look perfectly safe under the simplest possible check turn out to be unsafe once you account for how eyes actually move.

Where This Lives in the Code

This is why `comparison_metric/gates.py` applies both gates as independent, separately-required conditions (`apply_gates()` with both `gate_ghost_safety` and `gate_vignetting_safety`), rather than treating one as a shortcut proxy for the other.

6 A Real Bug Worth Understanding: When "Density" Stops Making Sense

6.1 The Problem

Packing density — how many holes fit per unit area — sounds like a simple thing to compute: count the holes, divide by the area they occupy. But "the area they occupy" is a genuinely tricky question once a requested pattern collapses down to very few actual holes (because the overlap-rejection step, described above, removed most of them).

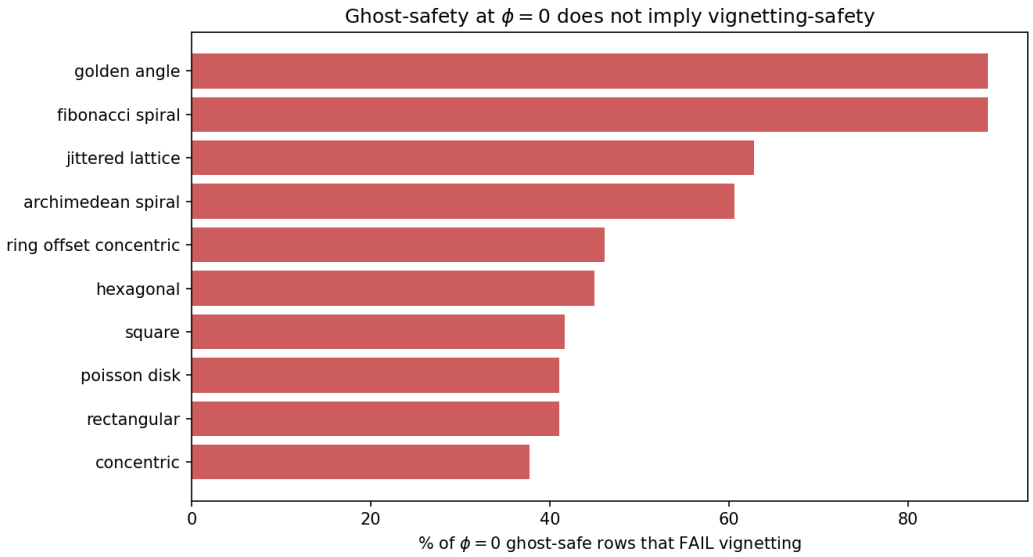


Figure 5: Fraction of straight-ahead ghost-safe designs, per geometry family, that fail the vignetting check once gaze angle is properly integrated.

Physical Intuition

Picture a formula that assumes a repeating grid pattern (a "unit cell" multiplied by how many times it repeats). If overlap-rejection has reduced a requested 20-hole pattern down to just *one* surviving hole, there is no repeating grid left to describe — the formula's own assumption has silently stopped applying, but nothing in the code was told to notice. The formula still returns a number; it is simply a meaningless one.

This defect was real and it mattered: it was caught only because a resulting design's reported density implied that its holes covered *more area than the space they were packed into* — a value greater than 100% open area, which is a plain physical impossibility and an excellent kind of bug to have a numerical assertion check for automatically, rather than relying on a human noticing an oddly large number.

6.2 The Fix, and the Lesson

The fix required two separate corrections, because two different families of geometry broke in two different ways:

1. For lattice families (square, hex, rectangular), the fix is a *feasibility check*: before trusting a unit-cell-based density, first confirm the requested spacing is even geometrically large enough to contain one hole. This check depends only on the ratio of hole size to spacing — it is completely independent of how many holes actually survived, which is exactly why simply requiring "at least 4 holes" *could not* have caught this case on its own.

2. For the ring/spiral/random families, the area was originally computed from the hole *centers* only, forgetting that each hole also has its own physical radius extending past its center — an omission proven with a real 7-hole test case where every hole was correctly non-overlapping, yet the naive area still came out too small. The fix adds each hole's own radius as a buffer around the boundary before computing area (a Minkowski/offset-buffer correction).

The broader lesson, useful well beyond this one bug: **a formula that is correct in the typical case can silently become meaningless in an edge case, and the only reliable way to catch that is to check the answer against a fact you know must always be true**

— here, that open area can never exceed 100%. Guessing at "reasonable-looking" thresholds (like a minimum hole count) is not a substitute for that kind of physical sanity check.

Where This Lives in the Code

The final, corrected logic lives in a single shared file, `array_geometry_lab/degeneracy_filter.py`, imported by every downstream script rather than re-implemented separately in each one — so a future new geometry family only has to get this right once.

7 Choosing a Winner: The Two Comparison Metrics

Once every candidate design has been checked against both safety gates, many designs pass. The question becomes: among the safe ones, which is actually best? "Best" needs a definition, and this project deliberately tried two different ones rather than assuming a single answer.

7.1 Metric 1: Maximum Packing Density

Simple: among every design that clears both safety gates, pick the one with the most holes per unit area. More holes per area means more open area overall (for a fixed hole size), which means more light gets through — directly useful once transmission/brightness is brought back into the picture later.

Physical Intuition

Why not just rank by *safety margin* instead? Because a design that is safe with enormous margin (holes spaced very far apart) is not necessarily useful — it might only fit a handful of holes. Ranking by margin alone would reward uselessly sparse designs. Density is the metric that actually reflects the practical goal.

7.2 Metric 2: The Pareto Front

Metric 1 collapses everything to a single winning point, which hides any trade-off that might exist between density and safety margin. Metric 2 instead reports the full *Pareto front* — every design for which no other design is simultaneously denser *and* safer. This shows the whole trade-off curve rather than one point on it.

In the current dataset, this front collapses to a single point at every myopia level — meaning, at the tested spacing resolution, safety in this project turned out to be close to binary (trivially safe versus clearly unsafe) rather than a graded trade-off. This was checked to be a genuine property of the data, not a bug in the Pareto logic itself, using a synthetic test dataset built specifically to confirm the dominance-checking code *can* produce a multi-point front when one actually exists.

Where This Lives in the Code

Both metrics live in `comparison_metric/metrics.py`, operating on whatever survives `gates.py`'s filter — deliberately kept as two separate, swappable functions so a future third metric does not require rewriting the comparison logic.

8 Closing the Loop: Confirming the Theory Against Real Physics

Everything above — the ghost formula, the vignetting geometry, both metrics — uses fast, *closed-form* formulas. They were validated once against real wave-optics simulation for a simple two-hole case (Section 2), but never checked against the specific, sometimes irregular, multi-hole patterns that actually won the comparison. A formula validated for a simple case is not automatically guaranteed to hold for a more complex one built from the same physics.

8.1 The Confirmation Step

The winning designs from Metric 1 were each propagated through the project's full multi-hole wave-optics engine (the same validated simulation machinery used for the original single-aperture optimization, not a shortcut) — and the resulting simulated image was inspected directly, with actual peak-detection on the simulated retina, rather than trusting the formula's prediction.

Physical Intuition

This distinction matters: the closed-form formula *predicts* where ghost images should land. The full simulation *measures* where they actually land, by literally propagating light through the eye model and looking at the resulting image. Confirmation means these two independent methods agree; it is not simply re-running the same formula a second time.

The result: **9 out of 9 finalist designs were confirmed** — the full simulation reproduced each finalist's predicted ghost-safety and vignetting-safety exactly, at every tested gaze angle. One genuinely useful side-finding also came out of this step: three of the nine finalists revealed that an older reference file (`blur_sweep.csv`) had been computed at a since-corrected, lower simulation resolution — caught because three finalists with *different* hole sizes reported the *identical* stale blur value to many decimal places, which is not something real, correctly-computed diffraction physics would ever do for different apertures. That reference file has since been regenerated at the corrected resolution.

Where This Lives in the Code

This confirmation pass lives in `array_geometry_lab_phase2_confirmation/`, importing (not copying) the original validated simulation engine directly — correctness matters more than speed here, since only a handful of finalist designs need checking, unlike the thousand-plus-candidate sweep that justified using fast closed-form copies everywhere else.

9 Summary: What a Design Needs to Pass

Step	Requirement
1	Hole size fixed by the existing acuity-optimal chart, per myopia level
2	Generate a candidate geometry (pattern, hole count, spacing)
3	**Gate 1:** no ghost-image pair violates the resolvable-separation limit
4	**Gate 2:** Gate 1 holds at every tested gaze angle, not just $\varphi = 0$
5	Among survivors, rank by packing density (Metric 1) or inspect the trade-off front (Metric 2)
6	Confirm the winner(s) against full wave-optics simulation, not just the formula

A design that passes all six steps has been checked, independently, at two different levels of physical detail (fast formula and full simulation), and is the point at which this project's theoretical work hands off to physical fabrication and measurement.